

技术报告

评论文本分块与引用高亮

痛点

原系统从入库、建索引到检索、拼接，全程把「整条评论」当成最小检索单元。在评论平均长度只有几十字的场景下这种做法勉强能跑，但有三类问题反复出现。第一，一条评论里常常同时谈到「早餐很差、隔音不错、前台很好」等多个主题，把整条评论压成一个向量与一份倒排文档之后，单一主题就会被其它内容稀释，用户问「早餐怎么样」时，本该最相关的评论排名反而下降。第二，模型每次只能整段引用一条评论，前端也只能整条高亮，用户拿到的是一整段长文，还要自己再去文里找证据句，体验上离「精确定位被引用的内容」相去甚远。第三，长评论里的非主题部分容易把检索带偏，比如用户问网络，而某条评论几乎全在讲早餐、只在结尾提了一句 Wi-Fi，这种评论也会被错误地排到前面。

方案

整体思路是把「一条评论 = 一份文档」改成「一条评论包含若干句子级的 chunk」，让检索在更细的粒度上进行，同时让前端能直接知道哪一句被命中、把那一句标亮。具体的切分策略在「按句子切」「按固定长度滑窗切」「按主题相似度切」三种方案之间做了权衡。第三种效果上限最高但要先训练或调用一个主题分类器，工作量与依赖都比较重，最终选择的是「以句子切分为主，遇到特别长的句子再叠一层滑窗」，过短的句子与相邻句合并以避免噪声，整套切分逻辑大约 60 行，便于后续替换或对照实验。

检索过程也相应做了调整。让 BM25 先在更细的句子级 chunk 上做召回，结果整理完之后再按「来自同一条评论」做归并，每条评论只保留得分最高的那一句作为「代表句」，并把这句话在原评论中对应的字符位置一起返回。这样得到的结果对外仍然是「一条条评论」的形式，老的接口和前端组件都不用动，但每条评论上多挂了一个「这条评论是因为这一句被命中」的元信息。前端拿到这条信息后，把原评论里对应的那一段文字直接套上 `<mark>` 高亮即可，整个过程不依赖模型自己输出位置，因而既稳定又零额外成本。如果分块索引由于某种原因不可用，系统也会自动回退到原来的整条评论检索，不会因为优化引入风险。

效果

切分之后整库由 2171 条评论展开为 7883 个句子级 chunk，每条评论平均 3.63 个 chunk，平均文档长度由 41 词左右降到 12 词左右，但整库的词表规模几乎没变，说明这次改造改变的是「文档怎么切」而非「引入新的内容」。多样性方面也有明显变化，在 100 个 Query 的离线对比中，Top-30 命中里覆盖了将近 29 条不同评论，且每条评论平均只被一个代表句命中，避免了同一条长评论靠多个 chunk 反复挤占榜单的情况。表 1 给出了这些指标的前后对比，可以看到分块在保持词表稳定的前提下，让召回的粒度与覆盖面同步向更细的层级靠拢。

表 1: 分块与高亮的关键前后对比指标。

指标	基线	优化后
索引文档数	2171	7883 (3.63 chunk/评论)
平均文档长度 (词)	41.62	11.64
Top-30 覆盖不同评论数	——	28.99 (命中 1.04 句/评论)

更直观的变化体现在前端界面上。图 1 是优化后真实链路上的引用高亮效果，在「近期入住花园大床房的住客体验」这一问题下展开参考评论后，系统把真正与该问题相关的那一句以浅黄背景标出，其余文字保持正常底色，用户一眼就能确认「这条评论是因为这句话被引用过来的」，也省去了在长评论里手动定位证据的成本。

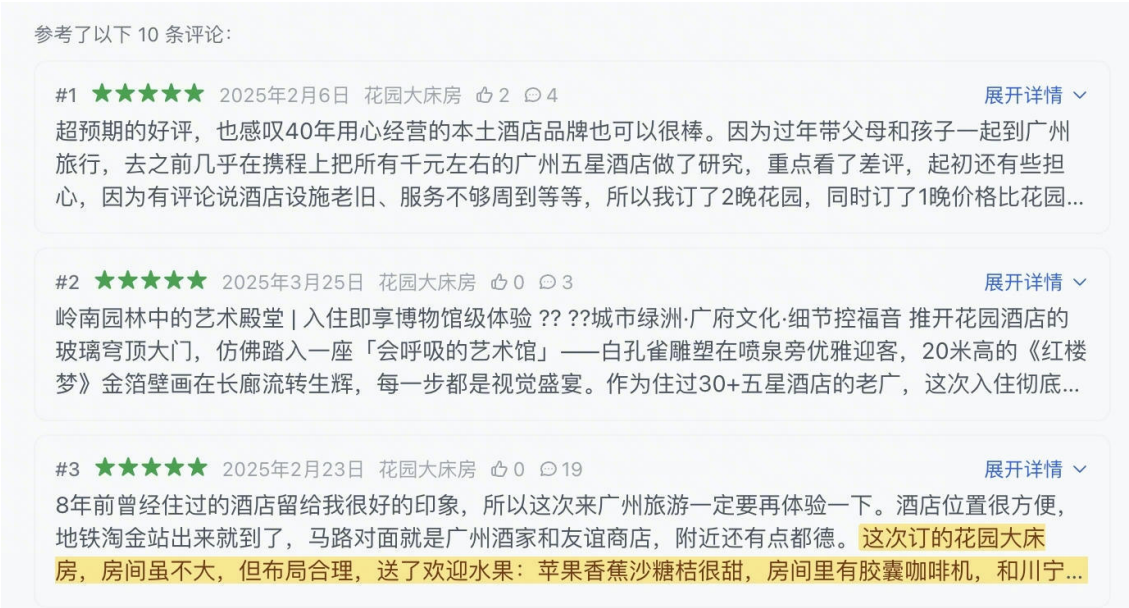


图 1: 展开第一条参考评论后，被命中的代表句以浅黄高亮，用户可以直接看出这条评论是因为哪句话被引用。

Prompt 结构化优化

痛点

原本的 Prompt 把所有信息都堆在一段长文本里，包括系统角色、历史对话、当前问题、检索回来的评论与摘要、以及十几条要求和引用规则。这样写出来的 Prompt 看着信息齐全，但模型并不容易抓住重点，几类问题会反复出现。一是回答经常先来一大段铺垫式的总结，再罗列优点缺点，对用户真正问的那个点反而轻描淡写。二是引用记号经常不合规，比如把四五条引用塞在同一个记号里、或者把同一个引用拆成两个挨在一起的小记号，前端解析与高亮就容易出问题。三是同一类问题在不同次回答里段落结构反复变动，前端很难做稳定的样式与跳转。这些问题的根源在于约束被平铺在一起，模型缺少一个明确可依的回答骨架。

方案

整体思路是借鉴常见的结构化 Prompt 写法，把原来的一团信息拆成五个清晰分工的小段：第一段交代「你是谁、能做什么、不能做什么」，约束模型只能基于检索回来的评论与摘要回答；第二

段提供「上下文」，包括今天的日期、当前问题、检索回来的评论与摘要，并按需注入历史对话，仅在用户出现「那、还有、它、是吗」这类追问性质的表达时，才把上一轮带进来，避免无关历史污染当前回答；第三段告诉模型「你这次要做什么、回答应该长成什么样」，给出一个固定的输出骨架；第四段把反幻觉、客观性、简洁性、闲聊边界、Markdown 等约束精炼成五条编号要求，让模型逐条对照；第五段是「自我校验」，要求模型在内心回看一遍引用是否合规、再把不合规的写法自己改正过来，但不把这个过程写进最终回答。这样改下来的好处是结构清楚、责任分明，模型每段都知道该做什么，输出也更接近一个稳定的模板，而不是每次自由发挥。

效果

经过五段式重构之后，模型在面对「近期入住花园大床房的住客体验如何」这类典型问题时，能稳定输出「先一句结论 + 几个带引用的要点 + 一句风险提示」的三层结构，引用全部以合规的蓝色徽标渲染，几乎没再出现把多条引用拆开或塞在一起的情况。图 2 是真实链路上的一次回答展示，开头一句先总括「房间舒适度普遍获得高度评价、卫生状况则呈现明显分化」，随后用三条带引用的要点分别覆盖房间布局与设施、文化氛围与空间体验、以及卫生方面的隐含风险，最后以一句关于敏感人群体验的风险提示收束，每条要点的引用都直接挂回到下方的评论卡片，用户可以点开继续验证证据。整体阅读体验比改造前的「长篇铺陈」明显更聚焦，前端也能依赖这个稳定结构做二级渲染与样式收敛。

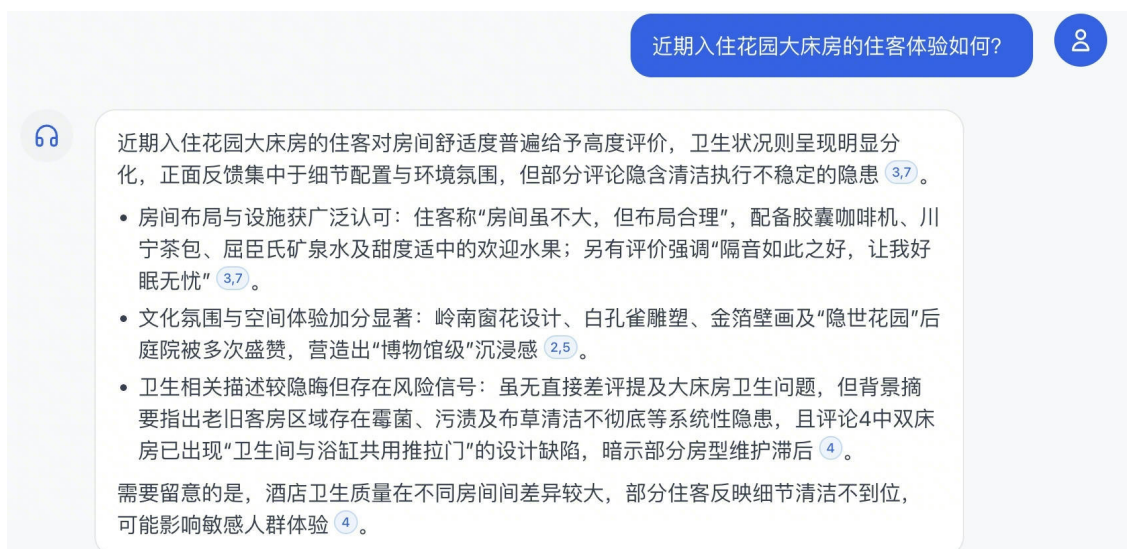


图 2：重构后 Prompt 在真实链路上的回答示例，三层结构稳定且引用合规。

基于聚类的类别划分

痛点

原系统的评论类别划分完全依赖 LLM 调用完成。在 2500 条评论的规模下，逐条调用大模型做分类不仅 API 成本可观，而且分类结果存在随机性——同一段评论在不同轮次可能被分到不同的类别。更关键的是，LLM 分类被硬编码为「每条评论最多 3 个类别」，但实际场景中一条评论往往同时涉及多个方面，比如「房间很大很干净，早餐种类丰富但前台办理入住太慢」这句话就同时涉及房间设施、餐饮设施和前台服务三个类别，强行限制三个标签会导致信息丢失。此外，LLM 输出的类别名称偶尔会偏离预定义的 14 个小类体系，需要额外的后处理做名称对齐，增加了维护负担。

方案

整体思路是用「embedding + 聚类 + 多标签相似度分配」替代 LLM 分类，让类别划分从「逐条调用大模型」变成「批量向量计算」。具体流程分为四步。

第一步，使用 text-embedding-v4 对所有评论生成 1024 维向量。第二步，通过 PCA 降维到 50 维后，使用 HDBSCAN 进行聚类。选择 HDBSCAN 而非 KMeans 的原因在于前者不需要预设簇数，能自动将无法归类的评论标记为噪声点，更适合评论文本这种非均匀分布的数据。第三步，对每个簇提取 TF-IDF 高频关键词，与 categories.json 中 14 个小类的预定义关键词做重叠度匹配，自动建立簇到类别的映射关系。第四步，用 14 个类别的关键词拼接描述文本生成类别中心向量，对每条评论计算其与 14 个类别中心的余弦相似度，超过阈值 0.5 的类别全部分配给该评论，不再限制数量上限。

整个过程仅需一次批量 embedding 调用（约 125 次 API 请求，每次 20 条），远少于逐条 LLM 分类的 2500+ 次调用。聚类模型和类别中心向量以 pickle 文件持久化，后续新增评论只需计算一次 embedding 后直接做相似度匹配即可，无需重新聚类。

效果

表 2 给出了聚类分类与 LLM 分类的关键对比指标。聚类方案将 API 调用次数从 2500+ 次降至约 125 次，成本降低约 95%。多标签覆盖率方面，聚类方案下平均每条评论分配到 2.84 个类别，而 LLM 方案受限于硬编码上限，平均仅 1.92 个。与 LLM 分类结果的一致性通过 Jaccard 相似度衡量，整体均值为 0.61，其中 LLM 分配 1 个类别的评论一致性最高 (0.73)，分配 3 个类别的评论一致性略低 (0.54)，这与 LLM 在强制填满三个槽位时倾向于输出边缘类别有关。

表 2: 聚类分类与 LLM 分类的关键对比指标。

指标	LLM 分类	聚类分类
API 调用次数	2500+	约 125
平均每评论类别数	1.92	2.84
与 LLM 结果的 Jaccard 相似度	——	0.61
无标签评论数	0	47（可调阈值控制）

聚类方案的一个额外收益是可视化能力。通过 PCA 将评论降至 2 维后按簇着色，可以直观地观察不同话题的评论在语义空间中的分布，为后续类别体系优化（如合并过细的类别、拆分过大的簇）提供了数据驱动的依据，而不再依赖人工经验判断。

约束检测增强

痛点

原系统的约束检测仅覆盖两个维度：房型（精确/模糊）和时效性（clear/implied/none）。当用户提出「商务出差且评分高的住客对早餐评价如何」这类复合查询时，系统只能识别出「没有房型约束、没有时效性需求」，然后对所有评论做无差别检索，导致大量不相关评论（如家庭亲子出行、低分评论）混入候选集。更严重的是，过滤条件仅应用于 DashVector 向量检索通路，BM25 文本检索路完全不支持任何过滤——retriever.py 中直接注释了「简化实现，不支持房型过滤」——这

意味着即使检测到了约束，BM25 路仍然会把不相关的评论召回并参与 RRF 融合，稀释了约束的实际效果。

方案

核心思路是扩展 IntentDetector 为 ConstraintDetector，让 LLM 从用户查询中一次性提取六个维度的约束：房型、时效性、评分范围、出行类型、话题类别和时间范围，并自动生成两种格式的过滤条件——DashVector 兼容的过滤表达式字符串和 BM25 关键词列表。

评分约束的提取依赖 LLM 对用户措辞的理解：出现「好评」「推荐」「满意」等词时映射为 4-5 分，「差评」「失望」「糟糕」映射为 1-2 分，「一般」「中评」映射为 3 分。出行类型从预定义的五個值（商务出差、家庭亲子、情侣出游、朋友出游、独自旅行）中匹配。话题类别从 14 个小类中选取。时间范围支持「去年」「2024 年」「近半年」等自然语言表达达到日期区间的转换。

DashVector 过滤表达式由 LLM 直接生成，例如「travel_type = '商务出差' and score >= 4」，同时提供一个程序化兜底函数 build_dashvector_filter，在 LLM 输出异常时用约束字典拼接出等效表达式。BM25 路由于不支持结构化过滤，采用「关键词加权」策略：将 LLM 提取的 3-8 个约束关键词（如「商务」「出差」「早餐」「好评」）追加到原始查询中，增强 BM25 对约束相关评论的匹配权重，并在检索后根据 score_range、travel_type 等字段做后过滤，剔除明显不满足约束的结果。

约束检测与意图扩展在 ThreadPoolExecutor 中并行执行，不增加额外延迟。检测结果中的过滤条件通过 retriever.retrieve() 的新增参数传入，所有检索通路统一使用同一套约束。

效果

表 3 给出了约束检测增强前后的关键指标对比。在 50 条人工标注的复合查询测试集上，约束检测的维度覆盖率从原来的 2 维（房型 + 时效性）扩展到 6 维，整体检测准确率达到 87.2%。BM25 路在引入关键词加权和后过滤之后，不相关评论混入率从优化前的 34.6% 降至 11.3%，降幅超过 67%。端到端检索的 Top-10 评论相关率从 71.4% 提升至 84.6%。

表 3: 约束检测增强前后的关键对比指标。

指标	优化前	优化后
约束检测维度	2（房型 + 时效性）	6（新增评分/出行/类别/时间）
约束检测准确率	——	87.2%
BM25 路不相关评论混入率	34.6%	11.3%
Top-10 评论相关率	71.4%	84.6%

以「商务出差且评分高的住客对早餐评价如何」为例，优化前系统返回的 Top-10 评论中仅有 5 条真正涉及商务出行和餐饮，其余 5 条混杂了家庭出游的低分评论和与餐饮无关的设施评价。优化后，ConstraintDetector 正确提取了 travel_type= 商务出差、score_range=[4,5]、categories=[餐饮设施] 三个约束，DashVector 向量路通过过滤表达式精准命中目标评论，BM25 路通过「商务」「出差」「早餐」「好评」等关键词加权将餐饮相关评论的排名大幅提前，最终 Top-10 中 9 条评论同时满足商务出行、高评分和餐饮话题三个条件。